

App1 - Raver's Paradise: Learning Concepts & Design Choices

Overview

Raver's Paradise is an intentionally vulnerable web application built for security education and CTF training. It simulates a real-world web application with a believable theme, realistic functionality, and a layered set of vulnerabilities that increase in complexity. Every design choice was made to teach a specific security concept in a realistic context without telegraphing the vulnerability to the player.

Design Philosophy

Realism Over Obviousness

The app is designed to look and feel like a real festival website. Vulnerabilities are not labelled or hinted at in the UI — players must approach it the same way a real penetration tester would, by exploring the application, reading URLs, testing inputs, and thinking critically about how the server might be processing their data. This mirrors real-world engagements where vulnerabilities are hidden inside otherwise normal-looking functionality.

Vulnerability Chaining

Several flags require chaining multiple vulnerabilities together rather than exploiting a single flaw in isolation. This reflects how real attacks work — a single vulnerability rarely leads directly to full compromise. The intended chains are:

- SQLi → Verb Tampering — The VIP token needed for flag7 is only discoverable by exploiting the SQL injection, meaning players must find and exploit flag2 before they can attempt flag7.
- Log Poisoning → LFI → RCE — Flag5 requires poisoning the log via the login page, then including the log via LFI, and finally using the resulting RCE to read a file. Three vulnerabilities must be understood and chained in the correct order.

- File Upload → Privilege Escalation — Flag6 gives the player a foothold on the server. Flag9 requires using that foothold to escalate privileges, teaching players that initial access is only the beginning.
- IDOR → Easter Egg — The archive page is visible in the nav but the easter egg file requires decoding, rewarding players who explore every part of the app.

Progressive Difficulty

The flags are roughly ordered by difficulty:

- Flags 1–3 are approachable for beginners — SQL injection and IDOR are well-documented and the payloads are relatively straightforward.
 - Flags 4–5 require understanding of how file paths and server-side includes work, and flag5 requires understanding a multi-step attack chain.
 - Flags 6–7 require knowledge of file upload bypass techniques, PHP execution, and HTTP methods — more intermediate concepts.
 - Flag 8 is a lateral thinking puzzle that rewards exploration and knowledge of classical ciphers.
 - Flag 9 is the hardest — it requires getting a shell, understanding Linux permissions, and knowing how to identify and exploit privilege escalation vectors.
-

Vulnerability Concepts

SQL Injection (Flags 1 & 2)

SQL injection is one of the most prevalent and well-understood web vulnerabilities, consistently appearing in the OWASP Top 10. The app demonstrates two distinct types:

Authentication bypass — The login query is built with raw string formatting, allowing an attacker to manipulate the query structure with a single quote and a comment sequence. This teaches players that SQL injection is not always about stealing data — sometimes the goal is simply to authenticate as a user you shouldn't have access to.

UNION-based data exfiltration — The notes search uses the same raw string approach, but here the goal is to extract data from tables the application never intended to expose. The `secrets` table is not referenced anywhere in the UI, teaching players the importance of enumerating the full database schema rather than stopping at the first result.

The use of SQLite keeps the setup simple and dependency-free while still being fully exploitable with standard SQL injection techniques.

IDOR — Insecure Direct Object Reference (Flag 3)

IDOR is one of the most common vulnerabilities found in real-world applications and one of the most frequently overlooked during development. The app exposes user IDs directly in the URL and performs no server-side check that the requesting user owns the resource they are asking for.

The design choice to redirect to `/notes?user_id=<id>` on page load was intentional — it puts the parameter in plain sight, hinting that it might be worth tampering with without explicitly telling the player what to do. The admin account is hidden from the dashboard community table, meaning players must deduce that user ID 1 is likely the admin by reasoning about how most applications are structured.

LFI — Local File Inclusion (Flag 4)

The artists page loads files based on a URL parameter with no path sanitization. This teaches players about directory traversal and the danger of passing user input directly to filesystem operations.

The default parameter `?file=artists.txt` is always present in the URL, which serves two purposes — it makes the parameter visible and exploitable without any special discovery, and it looks natural enough that a casual user might not notice it is interesting. The `../` traversal technique is a fundamental concept in web security and appears frequently in real-world applications that serve static files.

Log Poisoning + RCE (Flag 5)

This is one of the most instructive vulnerability chains in the app because it requires understanding three separate concepts and how they interact:

Log poisoning teaches that anything written to a log file is potentially dangerous if that file is ever interpreted rather than just read. The login page writes the raw username to the log — a realistic mistake that developers make when adding debugging or audit logging without considering what users might submit.

LFI as an execution vector teaches that file inclusion vulnerabilities are not just about reading sensitive files — if the included file contains executable code and the server interprets it, LFI becomes RCE. This is a critical conceptual leap for players to make.

RCE for post-exploitation teaches that code execution on the server is a means to an end, not the end itself. The flag is not displayed in the browser — players must use their RCE to navigate the filesystem and find it, mirroring real post-exploitation workflows.

File Upload + Reverse Shell (Flag 6)

Unrestricted file upload is a classic vulnerability that teaches players about the gap between client-side and server-side validation. The app blocks `.php` extensions but not `.phtml`, demonstrating that blocklist-based validation is inherently weaker than allowlist-based validation — there will always be another extension or bypass technique.

The double extension `revshell.png.phtml` is a real-world bypass technique. The `.png` component makes the filename look like an image to a casual observer while the `.phtml` extension is what the server actually uses to determine how to handle the file.

Using a real pentestmonkey-style PHP reverse shell rather than a simulated one teaches players how actual shells work — `fsockopen` for the socket connection, `proc_open` for spawning the shell process, and the shell upgrade technique using Python PTY and `stty` for a fully interactive terminal. These are skills that transfer directly to real engagements.

HTTP Verb Tampering (Flag 7)

The `/backstage` endpoint returns 403 on GET but accepts POST requests. This teaches players that HTTP methods are not just a UI concern — the server must explicitly handle and restrict each method independently.

The design choice to require the VIP token from the SQLi result creates a dependency chain that rewards thorough enumeration. Players who rush to test every endpoint without fully exploiting the SQL injection will be stuck on this flag. This mirrors real engagements where information gathered from one vulnerability is needed to exploit another.

The flag is returned as a plain text response to the POST request rather than rendered in a browser page, teaching players to read raw HTTP responses in curl or Burp Suite rather than relying on the browser to display everything.

Easter Egg (Flag 8)

The easter egg is designed as a puzzle that rewards lateral thinking and exploration rather than technical exploitation. The double encoding — Caesar cipher (ROT13) followed by base64 — teaches players to recognise common encoding schemes and apply them in the correct order.

The archive page is accessible from the navigation but easy to overlook, and the file contents are not immediately readable, teaching players to treat encoded or obfuscated data as a puzzle worth solving rather than noise to ignore.

Privilege Escalation — Sudo Baron Samedit (Flag 9)

Flag 9 teaches that getting a shell on a system is not the same as owning it. The flag file is owned by root with `600` permissions, meaning the initial shell obtained via the file upload vulnerability cannot read it directly.

The intended escalation path is Baron Samedit (CVE-2021-3156), a heap-based buffer overflow in `sudo` that allows any local user to escalate to root without knowing the root password. This was a critical real-world vulnerability that affected the vast majority of Linux systems for nearly a decade before being discovered and patched in January 2021.

This teaches players:

- How to enumerate privilege escalation vectors including sudo version checking
- What CVEs are and how to research and apply them
- That patching and keeping software up to date is a critical security control
- That the difference between a low-privilege shell and full system compromise can be a single unpatched binary

The realistic setup — a web app running on an unpatched system with a vulnerable sudo version — mirrors the kind of environment penetration testers encounter on real engagements.

Technical Design Choices

Flask + SQLite

Flask was chosen for its simplicity and transparency. The routing, templating, and database interaction are all visible and understandable without deep framework knowledge, making it easier for players to reason about how the vulnerabilities work at a code level if they choose to look at the source. SQLite keeps the database self-contained with no external service required.

Waitress as the WSGI Server

Flask's built-in development server was replaced with Waitress for stability and to avoid Windows Defender flagging the application. Waitress is a production-grade WSGI server that works on both Windows and Linux without requiring additional configuration.

Plaintext Passwords

Passwords are stored in plaintext in the database intentionally. This makes the SQL injection data exfiltration immediately rewarding — players who dump the users table get working credentials straight away, reinforcing why password hashing is a critical security control.

No CSRF Tokens

Forms across the app have no CSRF token protection, which is intentional for the CTF context. In a real application every state-changing form should include a CSRF token tied to the user's session.

PHP-CLI for Shell Execution

Using php-cli to execute uploaded `.phtml` files gives players a genuine PHP execution environment rather than a simulated one. This means real pentestmonkey reverse shells work exactly as they would against a real PHP web server, making the skills learned here directly transferable.

Neon EDM Theme

The theme was chosen to create an engaging and memorable environment. A believable and visually distinct application helps players stay immersed and think about it as a real target rather than an obvious training exercise. The raver theme also gives natural cover for features like file uploads, community notes, archive pages, and VIP access areas that might otherwise feel contrived. I also love EDM, so this was a very fun app to make.

App2 - DropZone CTF: Learning Concepts & Design Choices

Overall Design Philosophy

The app is built to look like a legitimate EDM community site rather than an obviously vulnerable training platform. This is intentional — real vulnerabilities exist in real applications that look normal. The neon rainbow aesthetic and EDM theme give it a believable identity while the vulnerabilities are woven naturally into the site's features rather than being labelled or obvious.

FLAG 10 — Stored XSS Cookie Theft

Learning Concept:

Cross-Site Scripting (XSS) occurs when user-supplied input is rendered as HTML without

sanitisation. Stored XSS is particularly dangerous because the payload persists in the database and fires for every user who views the page — including automated bots.

Why it works here:

Jinja2 escapes output by default, but the `| safe` filter was deliberately applied to `post.body` in `forum.html`, telling the template engine to trust the content. The `flag10` cookie was set with `httponly=False`, meaning JavaScript can read it via `document.cookie`. A real application should never store sensitive values in non-HttpOnly cookies.

Key lesson:

Never render user-supplied content as raw HTML. Never store sensitive values in JavaScript-accessible cookies. Defence in depth — even if XSS occurs, HttpOnly cookies prevent cookie theft.

FLAG 11 — Stored XSS + CSRF Chain

Learning Concept:

Cross-Site Request Forgery (CSRF) tricks an authenticated user's browser into making unintended requests. Combined with XSS, the attacker doesn't need to trick the user directly — the XSS payload acts as the delivery mechanism, running inside the victim's browser with their credentials attached.

Why it works here:

The `/settings` route accepts POST requests with no CSRF token — it blindly processes any password change request as long as a valid session cookie is present. The `fetch()` call in the XSS payload runs in the context of the bot's browser session, so the session cookie is automatically attached. The chain is XSS delivers the payload → fetch fires → settings route processes it → password changed.

Key lesson:

CSRF tokens bind a form submission to a specific user session, making it impossible to forge. Every state-changing endpoint should require one. XSS and CSRF are individually serious — chained together they become significantly more powerful.

FLAG 12 — Remote File Inclusion (RFI)

Learning Concept:

Remote File Inclusion occurs when an application fetches and processes content from a user-supplied URL without validation. The server makes the request, not the browser, so the attacker controls what the server executes.

Why it works here:

The visuals page was designed as a "load external visuals" feature, which is a believable real-world feature. The server fetches the URL with the `requests` library and passes the content through `subprocess` if it contains the `exec:` marker. There is no URL validation, no allowlist, and no sandboxing. The PHP execution path adds another layer — if the fetched content contains `<?php`, it is written to a temp file and executed with the PHP CLI.

Key lesson:

Never execute content fetched from user-supplied URLs. If external content must be fetched, validate the URL against a strict allowlist, never execute the response, and sandbox the process.

FLAG 13 — JWT None Algorithm Attack

Learning Concept:

JSON Web Tokens (JWTs) consist of a header, payload, and signature. The header specifies the signing algorithm. The `none` algorithm attack exploits implementations that trust the `alg` field in the header — if an attacker sets `alg: none` and removes the signature, a vulnerable server will skip verification entirely and accept any payload as valid.

Why it works here:

The `pyjwt` library's `decode` function was called with `verify_signature: False` and `algorithms=["HS256", "none"]` — explicitly allowing the none algorithm. The endpoint was hidden from the nav to encourage gobuster discovery, and a `/token` endpoint was provided to give players a valid token to inspect and understand before forging.

Design choice — gobuster discovery:

Hiding `/backstage` and `/token` from the UI teaches reconnaissance skills. Running gobuster against a real target is a standard first step, and wordlists like `dirb/common.txt` contain both `token` and `backstage` as entries.

Key lesson:

Never allow the `none` algorithm in JWT verification. Always specify a strict list of allowed algorithms and never trust the `alg` field from the token itself. Use a well-maintained library and keep it updated.

FLAG 14 — Server-Side Request Forgery (SSRF)

Learning Concept:

SSRF occurs when an application makes server-side HTTP requests to a URL controlled by the attacker. Because the request originates from the server, it can reach internal services that are not exposed to the public internet — localhost services, internal APIs, cloud metadata endpoints, and so on.

Why it works here:

The same visuals URL loader used for RFI also enables SSRF. The internal admin panel runs on `127.0.0.1:7777` — a port that is bound only to localhost and is completely unreachable from outside the machine. By pointing the visuals loader at `http://127.0.0.1:7777/`, the server makes the request to itself and returns the response to the attacker.

Design choice — shared RFI/SSRF surface:

Using the same endpoint for both RFI and SSRF is realistic — many real SSRF vulnerabilities exist in features designed for legitimate URL fetching like webhooks, link previews, and PDF generators. Players who find FLAG 12 via RFI are naturally positioned to discover FLAG 14 by experimenting with different URLs.

Key lesson:

Validate and restrict URLs before making server-side requests. Block requests to `127.0.0.1`, `169.254.169.254` (cloud metadata), and internal IP ranges. Use an allowlist of permitted hosts rather than a blocklist.

FLAG 15 — Root Flag via Privilege Escalation (PwnKit / CVE-2021-4034)

Learning Concept:

Post-exploitation privilege escalation involves leveraging a vulnerability in the operating system or installed software to elevate from a low-privilege user to root or SYSTEM after gaining initial access via the RFI shell.

PwnKit (CVE-2021-4034):

PwnKit is a local privilege escalation vulnerability in `pkexec`, a component of PolicyKit (`polkit`) that ships with virtually every major Linux distribution. It was present in every version of `pkexec` since its introduction in 2009 and was publicly disclosed in January 2022. The vulnerability is a memory corruption flaw in the way `pkexec` handles its argument list — an attacker with any local user access can exploit it to execute arbitrary code as root with no special permissions required.

Why it works here:

The RFI reverse shell gives the attacker code execution as the user running the Flask application — typically a low privilege account. PwnKit requires only local code execution to exploit, meaning any shell on a vulnerable unpatched system is sufficient to escalate to root.

Griz user makes it easier to get an area to use the vuln and also affording an ssh session instead.

Design choice:

Placing FLAG 15 in the root directory, and requiring a root shell to read it — it cannot be obtained via the `exec:` RFI tag or SSRF since those only return output from known commands, not arbitrary file reads from the attacker's perspective.

Key lesson:

Keep systems patched. Privilege escalation vulnerabilities in core system components like polkit can be exploited by any unprivileged user with local access. The principle of least privilege applies to the processes running your application — run web apps as dedicated low-privilege service accounts, not as root or your own user account.